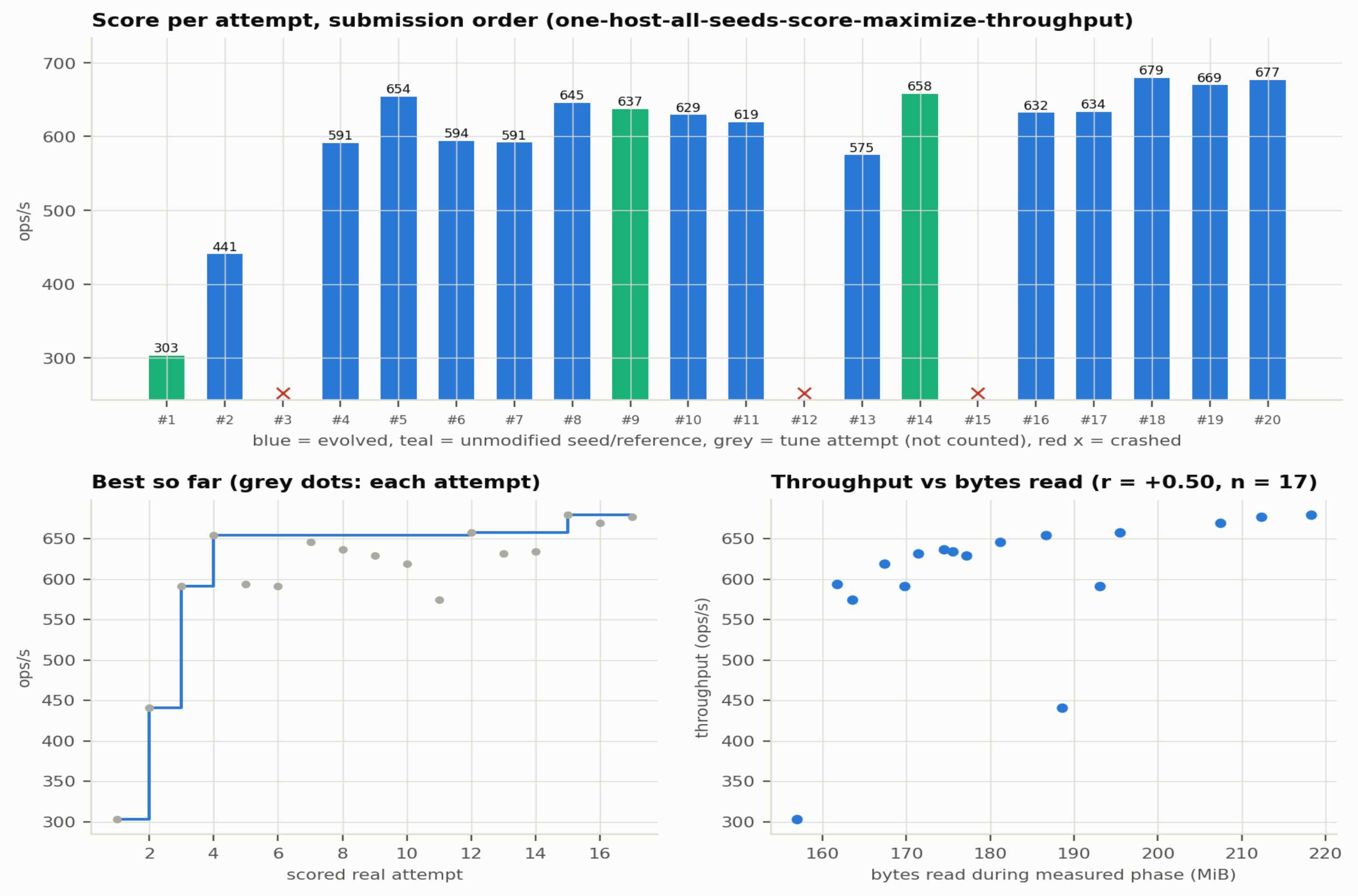
CORAL run 2026-10-20_173036 — one-host-all-seeds-score-maximize-throughput

all seven reference policies available; score = absolute measured throughput (ops/s). Evaluated on CloudLab host pc319.emulab.net (2-core, ~2 GB Xeon; absolute throughput is not comparable with the earlier c220g1 runs or with the other hosts).

real attempts	20 (cap 20) | scored 17, crashed 3 | tune 0
stop reason	max_real_attempts
wall time (agent logs)	0.49 h
best attempt	#18 e8ecbcfc score 679.3 — agent-evolved
best evolved	#18 e8ecbcfc score 679.3
unmodified seed (noop)	not submitted unmodified
LLM usage	37.36 USD, 46k output tokens, 102 turns in 21 sessions; gateway requests 159 {400: 41, 200: 118}


Score per attempt, submission order (one-host-all-seeds-score-maximize-throughput)

Best so far (grey dots: each attempt)

Throughput vs bytes read (r = +0.50, n = 17)

All attempts (submission order)

#	commit	class	score	tput ops/s	read MiB	refaults	wall s	design / title
1	30262b26	real =get_scan	303.2	303.2	157	15517	41	baseline: reference get_scan.c unchanged (scan-pid list + sampled LFU)
2	2a933640	real	440.5	440.5	189	23461	41	scan-first two-list LRU: scan-pid folios -> scan FIFO, others -> general LRU (move-to-tail on non-scan access); evict scan FIFO first then general hea
3	256c9c0c	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
4	f96a1fe6	real	591.4	591.4	193	23840	42	control retry: empty hooks + attaching loader (scan_pids map defined). Is attaching cache_ext with no victims ~1215 or slow?
5	80961586	real	653.9	653.9	187	22111	42	control: true noop (never attaches struct_ops) to calibrate today's node vs quoted 1210-1225; attached-empty was 591
6	38e09f3d	real	593.7	593.7	162	16632	40	two-list LRU + per-thread sequential-add scan detector (run>=4 consecutive pages in same inode -> scan FIFO), since scan_pids may be empty; skip rescu
7	31b2494f	real	591.5	591.5	170	18487	41	seq-detector two-list, drop folio_accessed hook entirely; scan FIFO evicts (PG_active folios rescued to general via continue_list), general list secon
8	35d0106b	real	645.4	645.4	181	20754	41	repeat: true noop (comment-only change vs 80961586) to measure noise vs attached ~591
9	0eaee1bf	real =s3fifo	636.7	636.7	174	19370	41	landscape: reference s3fifo.c unchanged (noop=654/645, attached-empty=591, best policy 594)
10	411f05f7	real	629.0	629.0	177	19983	40	S3-FIFO + scan detection (scan_pids or per-tid sequential adds): scan folios ignore scanner-thread accesses (no self-promotion from multi-key pages),
11	8bfe9878	real	618.9	618.9	167	17838	40	structural attempt 1/3 consumed-scan-first: S3-FIFO+scan-detect, plus done_list: scan folios move there once the scanner reads them and are evicted be
12	1e79f4fa	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
13	707eaae9	real	574.7	574.7	164	16797	41	instrumentation retry (u64->__u64 for skel): same policy as 8bfe987 + counters exported to policy_metrics files/stderr
14	f8d2b416	real =mglru	657.5	657.5	196	24545	40	landscape: reference mglru.c unchanged (noop 654/645, s3fifo 637; scan-aggressive variants all worse)
15	e0f2db84	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
16	8d6a71de	real	631.8	631.8	171	18778	40	retry (fixed bad edit): S3-FIFO + scan_pids-only scan marking + consumed-scan-first done_list; seq detector removed (misfires on GET readahead). Evide
17	52a50719	real	633.6	633.6	176	19756	41	MGLRU (ref 657.6, best) + scan_pids: scan-thread folios inserted at oldest gen (min_seq), scan-thread accesses don't bump refs
18	e8ecbcfc	real	679.3	679.3	218	29693	40	contrarian: MGLRU + scan-thread folios inserted at youngest gen (max_seq), refs counted normally. Every scan-demoting variant lost (mglru 658->634 w/
19	038076e0	real	669.5	669.5	207	27209	41	repeat of e8ecbcf (comment-only change): MGLRU + scan-thread folios at youngest gen, to confirm 679 vs mglru 658 is beyond noise
20	ce3fa61c	real	676.5	676.5	212	28248	41	MGLRU+scan@youngest (679/669) + non-scan new inactive folios always at min_seq (was min_seq+1 when >3 gens): sharpen scan-over-GET-new-page priority

Crashed attempts (no score; real ones still count toward the cap)

#3 256c9c0c (real): eval: compile failed: compile failed

#12 1e79f4fa (real): eval: compile failed: compile failed

#15 e0f2db84 (real): eval: compile failed: compile failed

Best evolved policy: #18 e8ecbcfc — score 679.3

Title: contrarian: MGLRU + scan-thread folios inserted at youngest gen (max_seq), refs counted normally. Every scan-demoting variant lost (mglru 658->634 w/ scan@oldest; s3fifo 637->619-632)

Closest available seed by line similarity: mglru 98%, get_scan 33%, mru 33%. Lines that differ from mglru are highlighted below.

```
// =============================================================================
// SECTION: BPF KERNEL CODE
// =============================================================================
// EVOLVE-BLOCK-START
// MGLRU: a BPF port of Linux's multi-generational LRU. Folios sit in generations (by age)
// and tiers (by access frequency); aging advances the generation counters and eviction
// takes from the oldest generation, with a ghost map feeding refault information back.
// captain-nemo: folios added by the scan thread (scan_pids) go to the YOUNGEST generation:
// demoting scan pages consistently cost throughput (evals 10/11/16/17), so test protecting them.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
#define INT64_MAX (9223372036854775807LL)

// #define DEBUG
#ifdef DEBUG
#define dbg_printk(fmt, ...) bpf_printk(fmt, ##__VA_ARGS__)
#else
#define dbg_printk(fmt, ...)
#endif

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define atomic_long_read(ptr) __sync_fetch_and_add(ptr, 0)
#define atomic_long_zero(ptr) __sync_fetch_and_and(ptr, 0)
#define atomic_long_store(ptr, val) __sync_lock_test_and_set(ptr, val)

#define DEFINE_MIN_SEQ(lrugen) \
    unsigned long min_seq = READ_ONCE(lrugen->min_seq)
#define DEFINE_MAX_SEQ(lrugen) \
    unsigned long max_seq = READ_ONCE(lrugen->max_seq)

//////////
// Maps //
//////////

#define MAX_NR_FOLIOS 4000000
#define MAX_NR_GHOST_ENTRIES 400000

struct folio_metadata {
    s64 accesses;
    s64 gen;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, MAX_NR_FOLIOS);
} folio_metadata_map SEC(".maps");

/////////////////
// Ghost Enties //
/////////////////

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    __uint(max_entries, MAX_NR_GHOST_ENTRIES);
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

static inline void insert_ghost_entry_for_folio(struct folio *folio, int tier) {
    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 ghost_val = (u8) tier;
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
            bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline int folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u8 *tier;
    tier = bpf_map_lookup_elem(&ghost_map, &key);
    if (tier == NULL) {
        return -1;
    }
    // TODO: handle non-ENOENT errors
    bpf_map_delete_elem(&ghost_map, &key);
    return (int)(*tier);
}

////////////////////////////////////////////////////////////////////////////////////
//  _  _______ ____  _   _ _____ _       ___ __  __ ____   ___  ____ _____ ____   //
// | |/ / ____|  _ \| \ | | ____| |     |_ _|  \/  |  _ \ / _ \|  _ \_   _/ ___|  //
// | ' /|  _| | |_) |  \| |  _| | |      | || |\/| | |_) | | | | |_) || | \___ \  //
// | . \| |___|  _ <| |\  | |___| |___   | || |  | |  __/| |_| |  _ < | |  ___) | //
// |_|\_\_____|_| \_\_| \_|_____|_____| |___|_|  |_|_|    \___/|_| \_\|_| |____/  //
////////////////////////////////////////////////////////////////////////////////////

// Constants

#define MAX_NR_TIERS 4
#define MIN_NR_GENS 2
#define MAX_NR_GENS 4
#define NR_HIST_GENS 1
#define MIN_LRU_BATCH 64
// Global policy metadata
struct mglru_global_metadata {
    struct bpf_spin_lock lock;
    unsigned long max_seq;
    unsigned long min_seq;
    s64 evicted[MAX_NR_TIERS];
    s64 refaulted[MAX_NR_TIERS];
    s64 tier_selected[MAX_NR_TIERS];
    s64 success_evicted;
    s64 failed_evicted;
    unsigned long avg_refaulted[MAX_NR_TIERS];
    unsigned long avg_total[MAX_NR_TIERS];
    unsigned long protected[MAX_NR_TIERS - 1];
    long nr_pages[MAX_NR_GENS];
};

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, int);
    __type(value, struct mglru_global_metadata);
    __uint(max_entries, 1);
} mglru_global_metadata_map SEC(".maps");

#define DEFINE_LRUGEN_void                                                      \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                     \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return;                                                         \
    }

#define DEFINE_LRUGEN_int                                                       \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                     \
```

```c
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return -1;                                                      \
    }

#define DEFINE_LRUGEN_bool                                                      \
    struct mglru_global_metadata *lrugen;                                   \
    int key__ = 0;                                                          \
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);       \
    if (!lrugen) {                                                          \
        bpf_printk(                                                     \
            "cache_ext: Failed to lookup lrugen metadata\n"); \
        return false;                                                   \
    }

// TODO: Hook these up with the eventual policy
// Helpers to update metadata

#define assert_valid_tier_0(tier_idx)                                  \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {                \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                  \
        return;                                                \
    }

#define assert_valid_gen_0(gen_idx)                                             \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                            \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return;                                                         \
    }

#define assert_valid_tier_1(tier_idx)                                  \
    if (tier_idx < 0 || tier_idx >= MAX_NR_TIERS) {                \
        bpf_printk("cache_ext: Invalid tier index %d\n", \
               tier_idx);                                  \
        return -1;                                             \
    }

#define assert_valid_gen_1(gen_idx)                                             \
    if (gen_idx < 0 || gen_idx >= MAX_NR_GENS) {                            \
        bpf_printk("cache_ext: Invalid gen index %d\n", gen_idx); \
        return -1;                                                      \
    }

inline void update_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
               s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->refaulted[tier_idx], delta);
}

inline void update_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx,
             s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->evicted[tier_idx], delta);
}

inline void update_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx,
              s64 delta)
{
    assert_valid_gen_0(gen_idx);
    __sync_fetch_and_add(&lrugen->nr_pages[gen_idx], delta);
}

inline void update_tier_selected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
              s64 delta)
{
    assert_valid_tier_0(tier_idx);
    __sync_fetch_and_add(&lrugen->tier_selected[tier_idx], delta);
}


inline s64 read_refaulted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->refaulted[tier_idx]));
}

inline s64 read_evicted_stat(struct mglru_global_metadata *lrugen, int tier_idx)
{
    assert_valid_tier_1(tier_idx);
    return max(0, atomic_long_read(&lrugen->evicted[tier_idx]));
}

inline s64 read_nr_pages_stat(struct mglru_global_metadata *lrugen, unsigned int gen_idx)
{
    assert_valid_gen_1(gen_idx);
    return max(0, atomic_long_read(&lrugen->nr_pages[gen_idx]));
}

// Invoke when promoting a folio in the eviction iteration
// See: https://github.com/cache-ext/linux-cachestream/blob/c22ffcac6b53ef4054483070fb902895ef10fd12/mm/vmscan.c#L4941-L4952
inline void update_protected_stat(struct mglru_global_metadata *lrugen, int tier_idx,
               s64 delta)
{
    __sync_fetch_and_add(&lrugen->protected[tier_idx - 1], delta);
}

// Gen lists
static __u64 mglru_lists[MAX_NR_GENS];

/******************************************************************************
 *                          PID controller
 ******************************************************************************/

/*
 * A feedback loop based on Proportional-Integral-Derivative (PID) controller.
 *
 * The P term is refaulted/(evicted+protected) from a tier in the generation
 * currently being evicted; the I term is the exponential moving average of the
 * P term over the generations previously evicted, using the smoothing factor
 * 1/2; the D term isn't supported.
 *
 * The setpoint (SP) is always the first tier of one type; the process variable
 * (PV) is either any tier of the other type or any other tier of the same
 * type.
 *
 * The error is the difference between the SP and the PV; the correction is to
 * turn off protection when SP>PV or turn on protection when SP<PV.
 *
 * For future optimizations:
 * 1. The D term may discount the other two terms over time so that long-lived
 *    generations can resist stale information.
 */

 // Avoid name conflict with kernel struct ctrl_pos if MGLRU is enabled
 struct ctrl_pos___x {
    unsigned long refaulted;
    unsigned long total;
    int gain;
};

static inline void read_ctrl_pos(struct mglru_global_metadata *lrugen, int tier,
              int gain, struct ctrl_pos___x *pos)
{
    pos->refaulted = lrugen->avg_refaulted[tier] +
             __sync_fetch_and_add(&lrugen->refaulted[tier], 0);
    pos->total = lrugen->avg_total[tier] +
             __sync_fetch_and_add(&lrugen->evicted[tier], 0);
    if (tier)
        pos->total += lrugen->protected[tier - 1];
    pos->gain = gain;
}

// Needs the LRU lock. Why???
static inline void reset_ctrl_pos(struct mglru_global_metadata *lrugen, bool carryover)
{
    int tier;
    bool clear = carryover ? NR_HIST_GENS == 1 : NR_HIST_GENS > 1;

    // lockdep_assert_held(&lruvec->lru_lock);

    if (!carryover && !clear)
        return;

    for (tier = 0; tier < MAX_NR_TIERS; tier++) {
        if (carryover) {
            unsigned long sum;

            sum = lrugen->avg_refaulted[tier] +
                  atomic_long_read(&lrugen->refaulted[tier]);
            WRITE_ONCE(lrugen->avg_refaulted[tier], sum / 2);

            sum = lrugen->avg_total[tier] +
                  atomic_long_read(&lrugen->evicted[tier]);
            if (tier)
                sum += lrugen->protected[tier - 1];
            WRITE_ONCE(lrugen->avg_total[tier], sum / 2);
        }

        if (clear) {
            atomic_long_zero(&lrugen->refaulted[tier]);
            atomic_long_zero(&lrugen->evicted[tier]);
            if (tier)
                WRITE_ONCE(lrugen->protected[tier - 1], 0);
        }
    }
}

static inline bool positive_ctrl_err(struct ctrl_pos___x *sp, struct ctrl_pos___x *pv)
{
    /*
     * Return true if the PV has a limited number of refaults or a lower
     * refaulted/total than the SP.
     */
    return pv->refaulted < MIN_LRU_BATCH ||
           pv->refaulted * (sp->total + MIN_LRU_BATCH) * sp->gain <=
               (sp->refaulted + 1) * pv->total * pv->gain;
}

///////////////////////////////////////
// Utils that use the PID controller //
///////////////////////////////////////

static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
{
    int tier;
    struct ctrl_pos___x sp, pv;

    /*
     * To leave a margin for fluctuations, use a larger gain factor (1:2).
     * This value is chosen because any other tier would have at least twice
     * as many refaults as the firsfirst tier.
     */
    read_ctrl_pos(lrugen, 0, 1, &sp);
    for (tier = 1; tier < MAX_NR_TIERS; tier++) {
        read_ctrl_pos(lrugen, tier, 2, &pv);
        if (!positive_ctrl_err(&sp, &pv))
            break;
    }

    return tier - 1;
}

// static inline int get_tier_idx(struct mglru_global_metadata *lrugen)
// {
//  // TODO: Swap this with the real implementation once we add refault tracking
//  // support.
//  return 0;
// }

static inline void folio_inc_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to inc refs but folio not found in map.\n");
        return;
    }
    __sync_fetch_and_add(&metadata->accesses, 1);
}

static inline int folio_lru_refs(struct folio *folio)
{
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata)
        return -1;

    return atomic_long_read(&metadata->accesses);
}

static inline unsigned int lru_gen_from_seq(unsigned long seq)
{
    return seq % MAX_NR_GENS;
}

static inline bool folio_test_active(struct folio *folio)
{
    return folio_lru_refs(folio) >= 2;
}

// Only counts up to 3.
static inline int order_base_2(int n)
{
    if (n <= 1) {
        return 0;
    } else if (n <= 3) {
        return 1;
    } else if (n <= 7) {
        return 2;
    } else if (n <= 8) {
        return 3;
    }
    return 3;
}

static inline int lru_tier_from_refs(int refs)
{
    /* see the comment in folio_lru_refs() */
    // kernel does some off-by-one calculation here, we remove the +1
    // return order_base_2(refs + 1);
    return order_base_2(refs);
}

static inline bool gen_within_limits(unsigned int gen)
{
    return 0 <= gen && gen <= MAX_NR_GENS;
}

static inline int get_nr_gens(struct mglru_global_metadata *lrugen)
{
    return lrugen->max_seq - lrugen->min_seq + 1;
}

static inline bool gen_almost_empty(struct mglru_global_metadata *lrugen,
                    int min_seq)
{
    int oldest_gen = lru_gen_from_seq(min_seq);
    int nr_folios = read_nr_pages_stat(lrugen, oldest_gen);
    int threshold = 4;
    return nr_folios <= threshold;
}

// TODO: This is supposed to run with a lock.
static inline bool lru_gen_add_folio(struct folio *folio, bool scan)
{
    unsigned long seq;
    if (folio_test_unevictable(folio)) {
        bpf_printk("cache_ext: Unevictable folio\n");
        return false;
    }

    DEFINE_LRUGEN_bool;
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    /*
     * There are four common cases for this page:
     * 1. If it's hot, i.e., freshly faulted in, add it to the youngest
     *    generation, and it's protected over the rest below.
     * 2. If it can't be evicted immediately, i.e., a dirty page pending
     *    writeback, add it to the second youngest generation.
     * 3. If it should be evicted first, e.g., cold and clean from
     *    folio_rotate_reclaimable(), add it to the oldest generation.
     * 4. Everything else falls between 2 & 3 above and is added to the
     *    second oldest generation if it's considered inactive, or the
     *    oldest generation otherwise. See lru_gen_is_active().
     */
    if (scan)
        seq = max_seq;
    else if (folio_test_active(folio))
        seq = max_seq;
    else if ((folio_test_reclaim(folio) &&
          (folio_test_dirty(folio) || folio_test_writeback(folio))))
        seq = max_seq - 1;
    else if (min_seq + MIN_NR_GENS >= max_seq)
        seq = min_seq;
    else
        seq = min_seq + 1;

    unsigned int gen = lru_gen_from_seq(seq);
    if (!gen_within_limits(gen)) {
        bpf_printk(
            "cache_ext: Invalid gen %d (min_seq=%lu, max_seq=%lu)\n",
            gen, min_seq, max_seq);
        return false;
    }

    // Update policy metadata
    struct folio_metadata metadata = { .accesses = 1, .gen = gen };
    __u64 key = (__u64)folio;
    int ret = bpf_map_update_elem(&folio_metadata_map, &key, &metadata,
                      BPF_ANY);
    if (ret != 0) {
        bpf_printk("cache_ext: Failed to save folio metadata\n");
        return false;
    }
    update_nr_pages_stat(lrugen, gen, folio_nr_pages(folio));

    // Update refaulted stats
    ret = folio_in_ghost(folio);
    if (ret >= 0) {
        int tier = ret;
        update_refaulted_stat(lrugen, tier, 1);
    }

    // lru_gen_update_size(lruvec, folio, -1, gen);
    ret = bpf_cache_ext_list_add(mglru_lists[gen], folio);
    if (ret != 0) {
        bpf_printk(
            "cache_ext: Failed to add folio to mglru_lists[%d]\n",
            gen);
        return false;
    }

    return true;
}

static inline bool should_run_aging(struct mglru_global_metadata *lrugen,
                    unsigned long max_seq)
{
    unsigned int gen;
    unsigned long old = 0;
    unsigned long young = 0;
    unsigned long total = 0;
    DEFINE_MIN_SEQ(lrugen);

    /* whether this lruvec is completely out of cold folios */
    if (min_seq + MIN_NR_GENS > max_seq) {
        return true;
    }

    unsigned long seq;
    int max_iter =  min(MAX_NR_GENS, max_seq - min_seq);
    for (int i = 0; i < max_iter; i++) {
        seq = min_seq + i;
        unsigned long size = 0;

        gen = lru_gen_from_seq(seq);

        size += max(read_nr_pages_stat(lrugen, gen), 0L);

        total += size;
        if (seq == max_seq)
            young += size;
        else if (seq + MIN_NR_GENS == max_seq)
            old += size;
    }

    /*
     * The aging tries to be lazy to reduce the overhead, while the eviction
     * stalls when the number of generations reaches MIN_NR_GENS. Hence, the
     * ideal number of generations is MIN_NR_GENS+1.
     */
    if (min_seq + MIN_NR_GENS < max_seq)
        return false;

    /*
     * It's also ideal to spread pages out evenly, i.e., 1/(MIN_NR_GENS+1)
     * of the total number of pages for each generation. A reasonable range
     * for this average portion is [1/MIN_NR_GENS, 1/(MIN_NR_GENS+2)]. The
     * aging cares about the upper bound of hot pages, while the eviction
     * cares about the lower bound of cold pages.
     */
    if (young * MIN_NR_GENS > total)
        return true;
    if (old * (MIN_NR_GENS + 2) < total)
        return true;

    return false;
}

static inline bool try_to_inc_min_seq(struct mglru_global_metadata *lrugen)
{
    // TODO: Check if the min_seq list is empty and if it is, increase min_seq.
    DEFINE_MIN_SEQ(lrugen);
    if (!gen_almost_empty(lrugen, min_seq)) {
        return false;
    }
    WRITE_ONCE(lrugen->min_seq, lrugen->min_seq + 1);
    reset_ctrl_pos(lrugen, true);
    return true;
}

static inline bool try_to_inc_max_seq(struct mglru_global_metadata *lrugen)
{
    // int prev, next;
    // spin_lock_irq(&lruvec->lru_lock);

    if (get_nr_gens(lrugen) == MAX_NR_GENS) {
        // Try to increase min_seq
        int ret = try_to_inc_min_seq(lrugen);
        if (!ret) {
            return false;
        }
    }
    // spin_unlock_irq(&lruvec->lru_lock);

    // TODO: Do we need this?
    /*
     * Update the active/inactive LRU sizes for compatibility. Both sides of
     * the current max_seq need to be covered, since max_seq+1 can overlap
     * with min_seq[LRU_GEN_ANON] if swapping is constrained. And if they do
     * overlap, cold/hot inversion happens.
     */
    // prev = lru_gen_from_seq(lrugen->max_seq - 1);
    // next = lru_gen_from_seq(lrugen->max_seq + 1);

    // for (type = 0; type < ANON_AND_FILE; type++) {
    //  for (zone = 0; zone < MAX_NR_ZONES; zone++) {
    //      enum lru_list lru = type * LRU_INACTIVE_FILE;
    //      long delta = lrugen->nr_pages[prev] -
    //               lrugen->nr_pages[next];

    //      if (!delta)
    //          continue;

    //      __update_lru_size(lruvec, lru, zone, delta);
    //      __update_lru_size(lruvec, lru + LRU_ACTIVE, zone, -delta);
    //  }
    // }

    reset_ctrl_pos(lrugen, false);

    // We don't use the timestamp metadata for our MGLRU
    /* make sure preceding modifications appear */
    WRITE_ONCE(lrugen->max_seq, lrugen->max_seq + 1);
    return true;

    // spin_unlock_irq(&lruvec->lru_lock);
}

/////////////////////////////////////////////////////////////
//   ____    _    ____ _   _ _____     _______  _______    //
//  / ___|  / \  / ___| | | | ____|   | ____\ \/ /_   _|   //
// | |     / _ \| |   | |_| |  _|     |  _|  \  /  | |     //
// | |___ / ___ \ |___|  _  | |___    | |___ /  \  | |     //
//  \____/_/   \_\____|_| |_|_____|___|_____/_/\_\ |_|     //
//                               |_____|                   //
/////////////////////////////////////////////////////////////


////////////
// Policy //
////////////


struct eviction_metadata {
    __u64 curr_gen;
    __u64 next_gen;
    __u64 iter_reached;
    __u64 tier_threshold;
};
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, u32);
    __type(value, struct eviction_metadata);
} mglru_percpu_array SEC(".maps");
```

```
struct eviction_metadata * get_eviction_metadata() {
    u32 key = 0;
    return bpf_map_lookup_elem(&mglru_percpu_array, &key);
}

void set_eviction_metadata(struct eviction_metadata *eviction_meta) {
    u32 key = 0;
    int ret = bpf_map_update_elem(&mglru_percpu_array, &key, eviction_meta, BPF_ANY);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to update eviction metadata\n");
    }
}

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, int);
    __type(value, bool);
    __uint(max_entries, 100);
} scan_pids SEC(".maps");

static inline bool is_scanning_pid(void)
{
    int pid = (int)(bpf_get_current_pid_tgid() & 0xFFFFFFFF);
    return bpf_map_lookup_elem(&scan_pids, &pid) != NULL;
}

inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio) {
        return false;
    }
    if (folio->mapping == NULL) {
        return false;
    }
    if (folio->mapping->host == NULL) {
        return false;
    }
    bool res = inode_in_watchlist(folio->mapping->host->i_ino);
    return res;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_int;
    WRITE_ONCE(lrugen->max_seq, MIN_NR_GENS + 1);
    for (int i = 0; i < MAX_NR_GENS; i++) {
        __u64 list_ptr = bpf_cache_ext_ds_registry_new_list(memcg);
        if (list_ptr == 0) {
                    bpf_printk(
            "cache_ext: Failed to allocate list for gen %d\n",
            i);
            return -1;
        }
        mglru_lists[i] = list_ptr;
    }
    return 0;
}


// MGLRU iteration function. Logic is mainly ported from sort_folio.
static int mglru_iter_fn(int idx, struct cache_ext_list_node *a)
{
    // We skip the following cases from sort_folio since we asssume we don't
    // accept these folios in this policy:
    // - Unevictable folios
    // - Swap-backed folios
    // - Promoted folios (these only appear through PTE accesses,
    //                    fd-accessed folios are promoted based on their tier)

    struct mglru_global_metadata *lrugen;
    int key__ = 0;
    lrugen = bpf_map_lookup_elem(&mglru_global_metadata_map, &key__);
    if (!lrugen) {
        bpf_printk(
            "cache_ext: Failed to lookup lrugen metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }

    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (!eviction_meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get eviction metadata\n");
        return CACHE_EXT_EVICT_NODE;
    }
    eviction_meta->iter_reached = idx;

    // Get folio metadata
    __u64 key = (__u64)a->folio;
    struct folio_metadata *meta =
        bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!meta) {
        bpf_printk("cache_ext: iter_fn: Failed to get metadata\n");
        // TODO: Maybe we should evict it instead?
        return CACHE_EXT_EVICT_NODE;
    }

    int tier_threshold = eviction_meta->tier_threshold;
    if (tier_threshold > MAX_NR_TIERS || tier_threshold < 0) {
        bpf_printk("cache_ext: Invalid tier threshold %d\n", tier_threshold);
    }
    // int tier_threshold = 2;
    int tier = lru_tier_from_refs(atomic_long_read(&meta->accesses));

    /* protected */
    if (tier > tier_threshold) {
        update_protected_stat(lrugen, tier, folio_nr_pages(a->folio));
        // promote to next gen
        // TODO: Update nr_pages stats
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }

    /* waiting for writeback */
    if (folio_test_locked(a->folio) || folio_test_writeback(a->folio) ||
        folio_test_dirty(a->folio)) {
        // promote to next gen
        int num_pages = folio_nr_pages(a->folio);
        update_nr_pages_stat(lrugen, eviction_meta->curr_gen, -num_pages);
        update_nr_pages_stat(lrugen, eviction_meta->next_gen, num_pages);
        atomic_long_store(&meta->gen, eviction_meta->next_gen);
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    DEFINE_LRUGEN_void;

    bool inc_max_seq_failed = false;
    bpf_spin_lock(&lrugen->lock);
    DEFINE_MIN_SEQ(lrugen);
    DEFINE_MAX_SEQ(lrugen);
    if (should_run_aging(lrugen, max_seq)) {
        if (!try_to_inc_max_seq(lrugen)){
            // Workaround: Not allowed to call bpf_printk under spinlock
            inc_max_seq_failed = true;
        }
    }
    if (max_seq - min_seq > MIN_NR_GENS)
        try_to_inc_min_seq(lrugen);
    // Read min/max seq again
    min_seq = READ_ONCE(lrugen->min_seq);
    max_seq = READ_ONCE(lrugen->max_seq);
    int oldest_gen = lru_gen_from_seq(min_seq);
    volatile unsigned int next_gen = (oldest_gen + 1) % MAX_NR_GENS;
    bpf_spin_unlock(&lrugen->lock);

    if (inc_max_seq_failed) {
        bpf_printk("cache_ext: Failed to increment max_seq\n");
    }

    int tier_threshold = get_tier_idx(lrugen);
    update_tier_selected_stat(lrugen, tier_threshold, 1);

    // Save eviction metadata for stats
    struct eviction_metadata ev_meta = {
        .curr_gen = oldest_gen,
        .next_gen = next_gen,
        .tier_threshold = tier_threshold,
    };
    set_eviction_metadata(&ev_meta);

    assert_valid_gen_0(next_gen);

    __u64 next_gen_list = mglru_lists[next_gen];
    __u64 oldest_gen_list = mglru_lists[oldest_gen];
    struct cache_ext_iterate_opts opts = {
        .continue_list = next_gen_list,
```

```c
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };


    int ret = bpf_cache_ext_list_iterate_extended(
        memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
    if (ret < 0) {
        bpf_printk("cache_ext: Failed to iterate list\n");
        return;
    }
    struct eviction_metadata *eviction_meta = get_eviction_metadata();
    if (eviction_meta == NULL) return;
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        min_seq = READ_ONCE(lrugen->min_seq);
        oldest_gen = lru_gen_from_seq(min_seq);
        next_gen = (oldest_gen + 1) % MAX_NR_GENS;
        __u64 next_gen_list = mglru_lists[next_gen];
        __u64 oldest_gen_list = mglru_lists[oldest_gen];
        struct cache_ext_iterate_opts opts = {
            .continue_list = next_gen_list,
            .continue_mode = CACHE_EXT_ITERATE_TAIL,
            .evict_list = CACHE_EXT_ITERATE_SELF,
            .evict_mode = CACHE_EXT_ITERATE_TAIL,
        };
        int ret = bpf_cache_ext_list_iterate_extended(
            memcg, oldest_gen_list, mglru_iter_fn, &opts, eviction_ctx);
        if (ret < 0) {
            bpf_printk("cache_ext: Failed to iterate list\n");
            return;
        }
    }
    s64 success_evicted = eviction_ctx->nr_folios_to_evict;
    s64 failed_evicted = max(0, eviction_ctx->request_nr_folios_to_evict - eviction_ctx->nr_folios_to_evict);
    __sync_fetch_and_add(&lrugen->failed_evicted, failed_evicted);
    __sync_fetch_and_add(&lrugen->success_evicted, success_evicted);
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        bpf_printk("cache_ext: Failed to evict requested number of folios: %d/%d. Used list idx %d, list ptr: %p. Iter reached %d\n",
                eviction_ctx->nr_folios_to_evict,
                eviction_ctx->request_nr_folios_to_evict,
                oldest_gen,
                oldest_gen_list,
                eviction_meta->iter_reached);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    lru_gen_add_folio(folio, is_scanning_pid());
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    folio_inc_refs(folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    if (!is_folio_relevant(folio)) {
        return;
    }
    DEFINE_LRUGEN_void;
    // Remove tracked metadata
    struct folio_metadata *metadata;
    __u64 key = (__u64)folio;

    metadata = bpf_map_lookup_elem(&folio_metadata_map, &key);
    if (!metadata) {
        bpf_printk(
            "cache_ext: Tried to delete folio metadata but not found in map.\n");
        return;
    }
    // Add ghost entry for refault detection
    int tier = lru_tier_from_refs(atomic_long_read(&metadata->accesses));
    insert_ghost_entry_for_folio(folio, tier);

    // Update generation page count
    update_evicted_stat(lrugen, tier, 1);
    update_nr_pages_stat(lrugen, metadata->gen, -folio_nr_pages(folio));
```

```c
    bpf_map_delete_elem(&folio_metadata_map, &key);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ===========================================================================
// SECTION: USERSPACE LOADER
// ===========================================================================
// Loader for the MGLRU policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0,
                      "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
        sa.sa_handler = sig_handler;
        if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
            perror("Failed to set up signal handling");
            return 1;
        }
    }

    // Parse command line arguments
    struct cmdline_args args = { 0 };
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, &args);

    // Validate arguments
    if (args.watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args.cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    // Does watch_dir exist?
    if (access(args.watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n",
            args.watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    char watch_dir_full_path[PATH_MAX];
    if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // TODO: Enable longer length
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    // Open skel
    skel = evo_policy_bpf__open();
    if (skel == NULL) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set watch_dir
    skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
    strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

    // Load programs
    {
        int scan_pids_fd = bpf_obj_get("/sys/fs/bpf/cache_ext/scan_pids");
        if (scan_pids_fd >= 0) {
            if (bpf_map__reuse_fd(skel->maps.scan_pids, scan_pids_fd))
                fprintf(stderr, "Failed to reuse scan_pids map fd\n");
            close(scan_pids_fd);
        } else {
            fprintf(stderr, "no scan_pids pin found\n");
        }
    }

    ret = evo_policy_bpf__load(skel);
    if (ret) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }

    // Initialize inode_watchlist map
    ret = initialize_watch_dir_map(args.watch_dir,
                       bpf_map__fd(skel->maps.inode_watchlist), false);

    // Attach cache_ext_ops to the specific cgroup
    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // Attach probes
    ret = evo_policy_bpf__attach(skel);
    if (ret) {
        perror("Failed to attach BPF programs");
        goto cleanup;
    }

    printf("evo_policy (MGLRU) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

Notes and caveats

- Hardware: pc319.emulab.net is a 2-core ~2 GB node; absolute numbers differ from other hosts and from the earlier c220g1 runs.
- Repeated evaluations of identical code vary by a few percent; differences of a few ops/s (or a few hundredths of tanh score) between attempts are within noise.
- No tune attempts were used.
- Best-by-score is a single measurement; re-measure before drawing conclusions about small gaps.